

Ejercicios: de la gramática a los structs
Unidad 0: Herramientas y preliminares

Eduardo Acuña Yeomans

Lenguajes de Programación
Semestre 2026-2

Lic. en Ciencias de la Computación
Universidad de Sonora

Sesión 4

Esta hoja no se entrega y no se califica. Se verifica de dos formas: el archivo tiene que compilar, y los árboles que construyas tienen que imprimirse igual que los de las soluciones.

La regla es la misma de siempre, y es lo único que hay que aplicar:

una producción, un struct cada no-terminal del lado derecho, un campo

Ninguna gramática de esta hoja es la del curso, a propósito. Y en ninguna hay que evaluar nada: aquí solo se representa.

Gramática A: expresiones booleanas

```
Bool ::= verdadero
      | falso
      | y(Bool, Bool)
      | o(Bool, Bool)
      | no(Bool)
```

1. Escribe los cinco structs. Usa los nombres `verdadero-exp`, `falso-exp`, `y-exp`, `o-exp` y `no-exp`, todos con `#:transparent`.

Sugerencia: una producción sin nada del lado derecho

`verdadero` no tiene ningún `Bool` adentro, así que su `struct` no lleva campos: (`struct verdadero-exp () #:transparent`). Los paréntesis vacíos son obligatorios. Para construir uno se escribe (`verdadero-exp`), también con paréntesis.

2. Construye en el REPL los árboles de estas dos cadenas:

```
y(verdadero, no(falso))
o(no(verdadero), y(falso, verdadero))
```

3. ¿Cuáles de estas cadenas *no* genera la gramática, y por qué?

(a) `no(verdadero, falso)` (b) `y(falso)` (c) `no(no(no(verdadero)))`

Gramática B: instrucciones para un robot

```
Prog ::= fin
      | avanza(Num, Prog)
      | gira-izq(Prog)
      | gira-der(Prog)
```

4. Escribe los cuatro **structs**, con los mismos nombres de la gramática.

Sugerencia: cuidado con esta

avanza lleva un **Num** y un **Prog**. Son dos campos, y no son la misma clase de cosa: el primero es un número y el segundo es el resto del programa. Fíjate en que la lista de instrucciones se acaba siempre con **fin**, igual que una lista de Racket se acaba con **()**.

5. Construye los árboles de:

```
avanza(3, gira-izq(avanza(2, fin)))
      gira-der(gira-der(fin))
```

6. ¿Cuántos **avanza** hay en el primero? ¿Y cuál es el **struct** que está más adentro de todos? Contéstalo mirando el árbol, sin correr nada.

Gramática C: rutas

```
Ruta ::= aqui
      | norte(Num, Ruta)
      | este(Num, Ruta)
      | repite(Num, Ruta, Ruta)
```

7. Escribe los cuatro **structs**. Esta gramática tiene una producción con **tres** elementos del lado derecho. Ninguna de las anteriores la tenía.

8. Construye el árbol de:

```
repite(3, norte(2, aqui), este(5, aqui))
```

Al revés

9. Alguien escribió estos **structs** y perdió la gramática. Recupérala: escribe la BNF que les corresponde, con la notación que usamos en clase.

```
(struct hoja (valor)           #:transparent)
(struct rama  (izq der)         #:transparent)
(struct etiq  (nombre sub)      #:transparent)
```

10. ¿Qué tienen en común todas las gramáticas de esta hoja, incluida la aritmética del curso? Fíjate en dónde aparece el no-terminal del lado izquierdo dentro de sus propias producciones, y en qué producción *no* aparece.

Soluciones

Clave de Respuesta: gramática A: booleanas

```
(struct verdadero-exp () #:transparent)
(struct falso-exp     () #:transparent)
(struct y-exp         (izq der) #:transparent)
(struct o-exp         (izq der) #:transparent)
(struct no-exp        (cuerpo)  #:transparent)
```

Ejercicio 2. Así los imprime el REPL:

```
> (y-exp (verdadero-exp) (no-exp (falso-exp)))
(y-exp (verdadero-exp) (no-exp (falso-exp)))

> (o-exp (no-exp (verdadero-exp)) (y-exp (falso-exp) (verdadero-exp)))
(o-exp (no-exp (verdadero-exp)) (y-exp (falso-exp) (verdadero-exp)))
```

Ejercicio 3. La (a) y la (b) no las genera. no lleva un solo Bool y en (a) tiene dos. y lleva dos y en (b) tiene uno. La (c) sí: no se puede anidar cuantas veces quieras, porque su campo es a su vez un Bool.

Clave de Respuesta: gramática B: el robot

```
(struct fin      () #:transparent)
(struct avanza   (pasos resto) #:transparent)
(struct gira-izq (resto) #:transparent)
(struct gira-der (resto) #:transparent)
```

```
> (avanza 3 (gira-izq (avanza 2 (fin))))
(avanza 3 (gira-izq (avanza 2 (fin))))

> (gira-der (gira-der (fin)))
(gira-der (gira-der (fin)))
```

Ejercicio 6. Hay dos avanza. El struct más adentro es (**fin**), y siempre lo va a ser: es la única producción que no lleva otro Prog dentro, así que es donde el árbol termina.

Clave de Respuesta: gramática C: rutas

```
(struct aqui    ()                #:transparent)
(struct norte   (cuantos resto)   #:transparent)
(struct este    (cuantos resto)   #:transparent)
(struct repite  (veces cuerpo resto) #:transparent)

> (repite 3 (norte 2 (aqui)) (este 5 (aqui)))
(repite 3 (norte 2 (aqui)) (este 5 (aqui)))
```

`repite` tiene tres campos porque su producción tiene tres cosas del lado derecho: un número y *dos* rutas: la que se repite y la que sigue después.

Clave de Respuesta: al revés y la pregunta final

Ejercicio 9. Los campos que son a su vez árboles se vuelven el no-terminal. Los que no, se quedan como `Num` o lo que corresponda:

```
Arbol ::= hoja(Num)
       | rama(Arbol, Arbol)
       | etiq(Nombre, Arbol)
```

Que `valor` sea un número y `nombre` un nombre no se puede deducir del código: los `structs` no dicen de qué tipo es cada campo. Cualquier respuesta razonable sobre esos dos vale. Lo que sí se deduce con certeza es *cuáles campos son árboles*, porque son los que hacen que la definición sea recursiva.

Ejercicio 10. Todas tienen al menos una producción donde el no-terminal aparece dentro de sí mismo (`y(Bool, Bool)`, `avanza(Num, Prog)`, `add(Exp, Exp)`) y al menos una donde **no** aparece (`verdadero`, `fin`, `aqui`, `Num`). Sin la primera clase no se podrían escribir programas de cualquier tamaño. Sin la segunda no se podría terminar ninguno. Es lo mismo que en una lista: `cons` y la lista vacía.

Y es la razón por la que toda función que recorra uno de estos árboles va a ser recursiva y va a tener un caso base, que es de lo que trata la sesión siguiente.